A Monad for Nonconstructivity

Jacob Prinz
Leonidas Lampropoulos

Abstract

Monads can be used to simulate a programming language feature in a language lacking it. For example, the 'Maybe' or 'option' monad can simulate exceptions. Monads can also be used to recover axioms lacking in an axiomatic system. For example, it is well known that in a constructive type theory, double negation is a monad within which one can use the law of the excluded middle.

In this paper, we define a monad to recover another major feature missing from constructive type theory: a convenient representation of nonconstructive functions. Much of traditional mathematics makes use of noncomputable functions, but most dependent type theories don't have a way to directly represent them. To solve this problem, our monad gives access to a definite description principle, allowing one to select a value by its unique description. By combining this monad with double negation, we are able to simulate nonconstructive features in a constructive type theory. Using these monads, we formalize the classical real numbers, define maps out of a natural definition of quotient types as equivalence classes, and define functions by general recursion. Our monad can be defined in any dependent type theory with extensionality principles, and all of our results are formalized in the Rocq prover.

1 Introduction

Dependent type theories used by theorem provers like Rocq, Agda, and Lean come with a built-in function type, $A \to B$, representing computable functions. These functions correspond to programs that are provably terminating, a restriction that is very useful: it ensures the consistency of the underlying type theory, and allows the functions to be used for both programming and proving. However, the traditional notion of a function as a mapping between inputs and outputs is often useful in verification, and includes functions that do not fit into this function type.

To represent such noncomputable functions, users usually resort to expressing them as relations, $A \to B \to \texttt{Prop}$, along with appropriate properties to restrict the relations to those that represent functions. However, this approach creates an extra competing notion of a function in type theory which is not compatible with convenient features like application and λ abstraction. For example, the composition of functions $f, g : T \to T$ is easy enough to express with the function type:

$$\lambda x.f\ (g\ x)$$

But expressing the same operation with relations $f, g : T \to T \to \mathit{Prop}$ is more difficult and requires an extra existential quantification (and a separate proof that the resulting relation is a function):

$$\lambda\ x\ z.\exists y, g\ x\ y \wedge f\ y\ z$$

It would be useful to be able to represent a larger class of functions while still using the function type. There is a standard approach for solving just this problem: define a *monad* that represents a larger class of values, and wrap it around the output type of the function [18].

Given a monad m, and $f, g : T \to m\ T$, we can define function composition using λ-abstraction and application, only paying the price of needing to rearrange our expression by binding intermediate computations:

$$\lambda x.bind\ (g\ x)\ f$$

Prior work has investigated the use of monads to represent various larger subsets of all mathematical functions. For example, coinductive monads or step counters can be used to represent semi-computable functions, or functions which correspond to programs that may not terminate [8, 9, 12]. For another example, the partial function classifier monad can be used to represent general partial functions [16]. We discuss these and other approaches in detail in Section 7. However, none of these existing monads in the literature represent the class of all mathematical functions.

While these subsets of all functions are widely useful, much of typical mathematics involves functions that fall in the fragment that existing approaches don't represent. For example, the Cauchy sequence construction of real numbers makes use of an infinite series of rational numbers satisfying a particular property. This infinite series is most naturally represented as a function $\mathbb{N} \to \mathbb{Q}$, and such a representation is used, for example, in the Rocq standard library's definition of Cauchy sequences [3]. However, this construction is only able to represent a restricted version of the real numbers, known as the constructive real numbers [3, 13], which

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

lack the usual completeness property. This is because in the proof of this completeness property, an infinite series is constructed that is not computable, and so does not fit into the function type, even with existing monadic approaches.

In this paper, we introduce a monad for nonconstructivity. If T is a type, then $[\![T]\!]$ represents the same set of elements but without the expectation that they are available constructively. The type of functions $A \to [\![B]\!]$ is the full class of all functions from A to B regardless of whether they correspond to a computation. Our monad can be defined in any dependent type theory with functional and propositional extensionality: $[\![T]\!]$ is the type of predicates over T that can be classically shown to uniquely specify a value.

One component of our novel monad is the well-known monad of double negation, $[T] = \neg\neg T$. Double negation also allows a kind of representation of nonconstructive values because the law of the excluded middle can be proven under it. However, $[T]$ is best thought of describing classical truth and falsity, as it contains at most one distinct element. Instead, we want to represent classical *values*, and so designed our $[\![T]\!]$ to have as many distinct elements as T.

Beyond its use to represent and reason about a larger class of functions, we show that our monad solves a problem with the natural definition of quotient types as equivalence classes. This traditional definition of quotients mostly works well in type theory with extensionality principles, except that one can't define maps out of the quotients. We show that maps out of these quotients can be defined as long as the output is into our monad.

Overall, our contributions are:

- We define a monad for nonconstructive values, which is definable in any dependent type theory with functional and propositional extensionality, and can be used to represent general functions (Section 2).
- We construct real numbers in terms of Cauchy sequences using our monad to represent the infinite series, giving the first construction of classical real numbers as Cauchy sequences in constructive type theory (Section 3).
- We show that our monad allows one to map functions out of quotients defined as equivalence classes, providing a useful quotient type with no extra axioms beyond extensionality (Section 4).
- We show that our monad can be used to define functions by general recursion. It has an advantage over some existing monads for general recursion in that the normal identity type works usefully with it (Section 5).

Finally, we discuss related work (Section 7). All of our contributions are formalized in the Rocq prover. We use abstract notation for dependent type theory throughout most of the paper, although we use Rocq syntax when we discuss general recursion.

2 The Monad

Our nonconstructivity monad depends on double negation, a well known monad for working with classical logic in constructive type theory, which we briefly review. If T is a proposition, define $[T]$ as the double negation of T:

$$[T] \coloneqq (T \to \texttt{False}) \to \texttt{False}$$

This type family is a monad, so we can define $return_P$ and $bind_P$, and these satisfy the expected monad properties:

$$return_P : T \to [T]$$
$$bind_P : [A] \to (A \to [B]) \to [B]$$

Although the law of the excluded middle is not provable in a constructive theory, it can be proven within this monad. This is done by wrapping the monad around the output of the function type in the law's statement:

$$lem : \forall P, [P + \neg P]$$
$$lem \coloneqq \lambda\, P\; x.x(inj_2\ (\lambda\, p.\ x\ (inj_1\ p)))$$

To define our nonconstructivity monad, we will need a type representing the universe of classical propositions. We define $CProp$ as the subset of $Prop$ that is of the form of a double negation:

$$CProp : Type$$
$$CProp \coloneqq \sum_{P:Prop} \exists P', P = [P']$$

We can convert $P : Prop$ into a $CProp$ by $[P]$ along with the trivial proof that $[P]$ is of the correct form. We overload our notation and use $[P] : CProp$ when $P : Prop$. We can convert a $CProp$ into a $Prop$ by taking the first projection; we elide the projection for readability.

Building on top of this monad for classical logic, we can now define our nonconstructivity monad. Given a type T, $[\![T]\!]$ is a dependent sum with three components, each with a carefully placed double negation: a subset S of T, a proof that S is inhabited, and a proof that all elements of S are equal:

$$[\![T]\!] \coloneqq \sum_{S:T\to CProp} [\exists t : T, S\ t] \wedge (\forall x \forall y, S\ x \to S\ y \to [x = y])$$

We refer to these three components as the subset, existence, and uniqueness. Crucially, the proof of existence is underneath a double negation, which will allow us to define elements of $[\![T]\!]$ when their existence is only known nonconstructively. The fact that the subset outputs a $CProp$ and that the uniqueness property has another double negation will allow a useful notion of equality for the type.

The existence and uniqueness properties are propositions, and so by proof irrelevance, which we get from propositional extensionality, they do not contribute information to the

value of this type. Only the subset can distinguish elements of the monad. Using functional and again propositional extensionality, these subsets are equal when they have the same element. Therefore, the elements of $[\![T]\!]$ correspond to the elements of T.

However, we can often construct an element of $[\![T]\!]$ when we could not for T. In fact, the definition of the nonconstructivity monad directly gives us a unique description principle: given a description of a unique T up to double negation, we can get an element of $[\![T]\!]$. This principle does not hold in general in type theory.

Rather than always directly working with this definition, most of the constructions in this paper will work in terms of a few useful properties that we outline below. We will give the full definitions and proofs for these properties Section 6.

First, we can define the monad operations for our type:

$$return : T \to [\![T]\!]$$
$$bind : [\![A]\!] \to (A \to [\![B]\!]) \to [\![B]\!]$$

In Section 6, we will show that these satisfy the three monad laws.

Next, we will need a way to prove properties of $[\![T]\!]$ in terms of elements of T. We can achieve that by proving an induction principle for our monad. This induction principle states that if we want to prove some classical property of $x : [\![T]\!]$, we have only to prove the property for the special case of $x = return\ t$. In addition, we get that t is in x's subset.

$$\begin{aligned} ind : (Q : [\![T]\!] \to Prop) &\to (x : [\![T]\!]) \\ &\to (\forall t, proj_1\ x\ t \to [Q\ (return_P\ t)]) \\ &\to [Q\ x] \end{aligned}$$

This induction principle only lets us prove classical propositions. Luckily, this isn't a problem for proving equality on our monad: if $a, b : [\![T]\!]$, then $[a = b] \to a = b$.

Finally, we will need a way to formulate propositions that depend on nonconstructive values. We can convert from nonconstructive propositions to normal ones:

$$toProp : [\![Prop]\!] \to Prop$$

We can reason about this function using the fact that $toProp\ (return\ P) = [P]$.

With all of these definitions in place, we can turn to applications of our monad: from defining real numbers, to solving a problem with a natural representation of quotient types, to a new way of encoding nontermination.

3 Real Numbers

The real numbers can not be defined directly in constructive type theory. The Cauchy sequence construction in particular runs into a problem with the representation of a noncomputable function. We first review the real numbers and these problems in constructive mathematics, and then describe how the nonconstructivity monad solves these problems.

3.1 Issues with Real Numbers in Type Theory

Formally, the real numbers are a set with two binary operations, addition and multiplication, that satisfy certain properties. First, these operations should satisfy the *field axioms*, which include things like commutativity of addition and existence of a multiplicative identity. Second, they should be totally ordered and satisfy *trichotomy*, meaning that given any two reals one should be able to decide if they are equal, or if not then which is greater. Finally, they should also satisfy the *Dedekind completeness property*, which states that any nonempty bounded set has a least upper bound.

However, the total ordering and least upper bound properties together pose a problem in constructive mathematics. Given any proposition P, we can construct a nonempty and bounded set of reals whose least upper bound is 0 if P is false and 1 if P is true, by only including 1 when P is true: $\{x \mid x = 0 \vee (P \wedge x = 1)\}$. But the total ordering property implies that we can decide if a real number is equal to or greater than 0, and therefore that we can also decide the truth of arbitrary propositions. To get around these problems and define the real numbers in a constructive type theory, we will only prove trichotomy up to a double negation.

Even given this concession that total ordering will be proven only up to double negation, our nonconstructivity monad is needed to solve a more difficult problem that arises with the standard Cauchy sequence construction: A Cauchy sequence is a converging sequence of rational numbers, and two Cauchy sequences are equivalent when they converge to the same value. If we represent a Cauchy sequence as a function in type theory $\mathbb{N} \to \mathbb{Q}$, we can prove all of the field axioms and the total ordering axiom holds up to double negation. However, the proof the of the least upper bound property fails, even up to double negation.

To see why we can't construct a Cauchy sequence for a least upper bound constructively, and how our nonconstructivity monad will solve the problem, we first review the standard classical construction. Suppose that we have a nonempty and bounded set $S \subset \mathbb{R}$, and two rational numbers $l_0, u_0 : \mathbb{Q}$ where u_0 is a upper bound of S, and l_0 is not an upper bound. We will construct two Cauchy sequences $\{u_i\}$ and $\{l_i\}$ which will approach the least upper bound from above and below using a binary search. For each i, we consider the midpoint $(u_i + l_i)/2$. If this midpoint is an upper bound, then we use it for the next upper value u_{i+1}, and otherwise for the next lower value l_{i+1}, while the other value remains unchanged from the previous index. In other words, we perform a binary search to bring the values of our two sequences ever closer to the true value of the least upper bound.

The proof continues by proving that these two sequences converge to the least upper bound of the set, but this first part of the construction already poses an issue in constructive mathematics. The property checked at each step of the definition of u_i and l_i, that the midpoint is an upper bound of S, is undecidable! Therefore, the classic proof invokes the law of the excluded middle at each step in defining the two sequences. If we represent Cauchy sequences as functions $\mathbb{N} \to \mathbb{Q}$, we have no hope of defining this construction.

3.2 A Nonconstructive Series Using Our Monad

Instead, we will represent Cauchy sequences as functions $\mathbb{N} \to [\![\mathbb{Q}]\!]$. The function constructed in the proof fits in this type, and we can define it naturally using a new combinator for our monad. We define 'propositional if', which behaves just like an if expression, except that the condition is a proposition instead of a boolean value and the output is inside the monad:

$$Pif : \texttt{Prop} \to T \to T \to [\![T]\!]$$
$$Pif\ P\ x\ y \coloneqq (\lambda t.[(P \wedge x = t) \vee (\neg P \wedge y = t)], \_, \_)$$

The uniqueness proof is straightforward, since we can't have both P and $\neg P$. The existence proof makes good use of the fact that our nonconstructivity monad only requires existence up to a double negation by calling the law of the excluded middle on P. We can also prove two theorems which provide defining equations for this function:

$$P \to Pif\ P\ x\ y = x$$
$$\neg P \to Pif\ P\ x\ y = y$$

Using propositional if, we can define a function that produces the two sequences in type theory using our monad very naturally. In the recursive step defining (u_{i+1}, l_{i+1}), it uses *bind* to get (u_i, l_i), and uses a propositional if to check if the midpoint is an upper bound. Given a predicate for checking if a real bounds a set *is_upper_bound*, we can define a function which produces these two sequences:

$$\begin{aligned}
&\texttt{u_and_l} : \mathbb{N} \to [\![\mathbb{Q} \times \mathbb{Q}]\!] \\
&\texttt{u_and_l}\ 0 \coloneqq (u_0, l_0) \\
&\texttt{u_and_l}\ (succ\ i) \coloneqq \\
&\qquad bind\ (\texttt{u_and_l}\ i)\ \lambda\ (u_i, l_i). \\
&\qquad \texttt{let}\ mid\ \coloneqq\ (u_i + l_i)/2\ \texttt{in} \\
&\qquad Pif\ (is_upper_bound\ S\ mid)\ (mid, l_i)\ (u_i, mid)
\end{aligned}$$

3.3 Full Construction of Cauchy Sequences

Confident that our nonconstructivity monad can handle the nonconstructive part of the proof, we proceed to define Cauchy sequences and prove that they satisfy all of the real number axioms. We define a type of Cauchy sequences, which is a record consisting of the sequence and the cauchy property. The cauchy property is standard and enforces that the sequence is converging by requiring that for all small values ε, there is some index of the sequence beyond which all of the values are within ε of each other. However, we need to place the existence quantifier underneath a double negation monad, because in the classical real numbers this value is not known constructively. We also need to use some of our monad combinators, *bind*, *return*, and *toProp*, to formulate the condition on the sequence values.

$$\begin{aligned}
cauchy \coloneqq\ &\{seq : \mathbb{N} \to [\![\mathbb{Q}]\!], \\
&\quad property : \forall \varepsilon > 0, [\exists N, \forall i, j \geq N, \\
&\qquad toProp\ (bind\ (seq\ i)\ \lambda\ s_i. \\
&\qquad\quad bind\ (seq\ j)\ \lambda\ s_j. \\
&\qquad\quad return\ (|s_i - s_j| \leq \varepsilon))]\}
\end{aligned}$$

We also define define an ordering $\leq_c$ for our cauchy numbers. This follows the standard mathematical definition, but again we need to be careful to place a double negation monad to remove any constraints imposed by constructivity.

$$\begin{aligned}
&\leq_c : cauchy \to cauchy \to cauchy \\
&x \leq_c y \coloneqq \forall \varepsilon : \mathbb{Q}, \varepsilon > 0 \to \\
&\qquad [\exists N : \mathbb{N}, \forall n, N \leq n \to \\
&\qquad\ toProp(bind(x\ n)\ \lambda x_n. \\
&\qquad\qquad bind(y\ n)\ \lambda y_n. \\
&\qquad\qquad return((x - y) \leq \varepsilon))]
\end{aligned}$$

Using this inequality relation, we can define $x =_c y$ as $x \leq_c y$ and $y \leq_c x$.

The definition of the other operations on Cauchy sequences follow the standard definitions, except that we need to use our monad combinators. For example, we can define addition of Cauchy sequences:

$$\begin{aligned}
&add : cauchy \to cauchy \to cauchy \\
&add\ a\ b \coloneqq \{seq \coloneqq \lambda i.bind\ (seq\ a\ i)\ \lambda a_i. \\
&\qquad\qquad\qquad bind\ (seq\ b\ i)\ \lambda b_i. \\
&\qquad\qquad\qquad return\ (a_i + b_i) \\
&\qquad\qquad property \coloneqq \cdots\}
\end{aligned}$$

All of the proofs about Cauchy sequences follow the classical proofs. We will show a part of the proof of commutativity of addition in order to demonstrate the use of the reasoning principles for the nonconstructivity monad introduced in Section 2.

To prove that *add* is commutative up to cauchy equivalence, given $a, b : cauchy$, it is sufficient to prove the stronger property that for all $i : \mathbb{N}$,

$$\begin{aligned} &toProp\ (bind\ (seq\ (add\ a\ b)\ i)\ \lambda\ s_{ab}.\\ &\quad bind\ (seq\ (add\ b\ a)\ i)\ \lambda\ s_{ba}.\\ &\quad return\ (s_{ab} = s_{ba})) \end{aligned}$$

Expanding the definition of addition, and using the monad laws to simplify the nested *bind*s, this becomes:

$$\begin{aligned} &toProp\ (bind\ (seq\ a\ i)\ \lambda\ a_i.\\ &\quad bind\ (seq\ b\ i)\ \lambda\ b_i.\\ &\quad bind\ (seq\ b\ i)\ \lambda\ b'_i.\\ &\quad bind\ (seq\ a\ i)\ \lambda\ a'_i.\\ &\quad return\ (a_i + b_i = b'_i + a'_i)) \end{aligned}$$

Now we can use our induction principle for the nonconstructivity monad on both *seq a i* and *seq b i*. This gives us values $x, y : \mathbb{Q}$, and replaces *seq a i* with *return x*, and *seq b i* with *return y* in the goal:

$$\begin{aligned} &toProp\ (bind\ (return\ x)\ \lambda\ a_i.\\ &\quad bind\ (return\ y)\ \lambda\ b_i.\\ &\quad bind\ (return\ y)\ \lambda\ b'_i.\\ &\quad bind\ (return\ x)\ \lambda\ a'_i.\\ &\quad return\ (a_i + b_i = b'_i + a'_i)) \end{aligned}$$

Finally, we can simplify using the first monad law:

$$toProp\ (return\ (x + y = y + x))$$

We can now use our property on *toProp* to simplify this to a statement that addition commutes on rational numbers:

$$[x + y = y + x]$$

After calling $return_P$, we use the Rocq standard library proof of rational addition commutativity.

Our formalization includes the entire development of real numbers from Cauchy sequences: the definitions of equivalence, less than, addition and multiplication, additive and multiplicative inverses, proofs that these operations respect the equivalence $=_c$, the field axioms, and the total ordering and least upper bound properties.

As we mentioned at the beginning of the section, it is inevitable that the total ordering property is only true up to double negation.

$$(x\ y : cauchy) \to [x \leq_c y \vee y \leq_c x]$$

However, there is an additional gap between our real numbers and the ideal axiomatization: the least upper bound property is also only known classically. This is because the in the construction of the least upper bound we carried out earlier in our definition `u_and_l`, we need to start with u_0, a rational that bounds the set, and l_0, a rational that is not a bound. However, because our definition of Cauchy sequences includes a double negation surrounding the existence of the index, we can only prove the existence of these rationals classically. We are able to prove this statement where the existence of the least upper bound is wrapped in a double negation:

$$\begin{aligned} &(S : cauchy \to Prop)\\ &\to [\exists r, Sr] && \text{(nonempty)}\\ &\to [\exists b, is\_upper\_bound\ S\ b] && \text{(bounded)}\\ &\to [\exists\ lub, is\_upper\_bound\ S\ lub\\ &\qquad \wedge (\forall b', is\_upper\_bound\ S\ b' \to lub \leq_c b')] \end{aligned}$$

In the next section, we will fix this problem by placing our Cauchy sequences into a quotient type.

4 Quotient Type

Although many dependent type theories don't have a quotient type, quotients are essential to formalizing traditional mathematics. The Lean theorem prover, for example, has axiomatized quotient types, and they are used extensively to represent traditional mathematical definitions [1, 15].

However, there is a natural way to define quotient types more directly in dependent type theory given functional and propositional extensionality by using the classic construction in terms of equivalence classes. We will show that with a few careful placements of the double negation monad this simple definition of quotients can be fixed to satisfy all of the desired properties, at least up to our monads.

Given a type T and an equivalence relation R, we define the quotient type T/R as the type of equivalence classes of R. More specifically, the type is a triple of three parts: a subset of T, a proof that the subset is nonempty, and a proof that the subset is closed under R.

$$T/R := \sum_{S:T\to CProp} [\exists t, S\ t] \wedge (\forall xy, Sx \to [R\ x\ y] \iff S\ y)$$

By proof irrelevance, the only component of these quotients that is relevant to equality is the subset S. And by functional and propositional extensionality, two of these subsets are equal exactly when they have the same elements.

Quotients defined this way already directly satisfy most of the desired properties that a quotient should, up to double negation. We first show these properties which work, and then show how our nonconstructivity monad can help to define the last property that is otherwise broken.

Properties Already Derivable. First, we can define a function that maps an element into its equivalence class. This function sends an element to the subset of elements which are related to it by R. That this set is inhabited follows from

the reflexivity of R, and that it is closed under R follows from transitivity and symmetry.

$$mk : T \to T/R$$
$$mk\ t \coloneqq (\lambda\ x.[R\ t\ x], \_, \_)$$

These quotients satisfy soundness and completeness properties. If two elements are related, then their mappings into the quotient are equal by the fact that R is an equivalence relation combined with extensionality. Similarly, if the mappings of two elements are equal, then they are related. However, we only get this completeness property under the double negation monad.

$$\forall x\ y, R\ x\ y \to mk\ x = mk\ y$$
$$\forall x\ y, mk\ x = mk\ y \to [R\ x\ y]$$

These quotients also satisfy an induction principle, which says that we may assume that all elements of a quotient are of the form $mk\ x$ when reasoning. This is provable because as the output of the induction principle is a proposition, we can access the propositional proof that the set is inhabited.

$$ind_Q : (P : T/R \to Prop) \to (\forall a, [P\ (mk\ a)])$$
$$\to \forall q, [P\ q]$$

The induction principle only outputs propositions under a double negation. This is particularly a problem when we want to prove equalities using the induction principle, and so we have designed our quotients to admit the following property that lets us get a direct equality from one under a double negation. Given $x, y : T/R$,

$$[x = y] \to x = y$$

Lift with Nonconstructivity Monad. Even the simple definition of quotient types with no instances of the double negation monad in it will satisfy all of the properties described so far, modulo placements of double negations. However, there is one important quotient property that could not be derived for these quotient types without our nonconstructivity monad: the ability to map a function out of the quotient. Ideally, it should be possible to map out of a quotient by a function which respects the underlying equivalence relation. This lifting operation would have the following type:

$$(f : A \to B) \to (\forall xy, Rxy \to fx = fy)$$
$$\to A/R \to B$$

However, our simple definition of quotients in terms of equivalence classes does not admit this operation, because the value is hidden inside a proposition and so is not available for computation.

While we can't define a lifting operation with that type, we can derive one which outputs into our nonconstructivity monad with the following type. It differs from the ideal one only in the use of the monad around the output:

$$lift : (f : A \to [\![B]\!]) \to (\forall xy, Rxy \to fx = fy)$$
$$\to A/R \to [\![B]\!]$$

To implement $lift$, we need to provide a singleton subset of B given our equivalence class in A. This is done easily: we can define it as the image of f over our subset of A. That this is a singleton follows from the fact that f respects the equivalence relation.

$$lift\ f\ \_\ (S, nonempty, closed)$$
$$\coloneqq (\lambda\ b.\exists a, S\ a \wedge proj_1\ (f\ a)\ b, \_, \_)$$

Finally, we are able to prove a theorem which allows us to reason about the value of $lift$ up to propositional equality. Lifting a function over the equivalence class of an element is equal to simply applying the function to the element directly:

$$lift\_eq : lift\ f\ r\ (mk\ x) = fx$$

This lifting operation only outputs into the nonconstructivity monad. However, we can get around this problem when the output is into a quotient. We can define a function that unwraps our nonconstructivity monad from around a quotient type:

$$unwrap : [\![T/R]\!] \to T/R$$

We also prove a defining equation that lets us work with $unwrap$:

$$unwrap\ (return\ x) = x$$

4.1 Quotient of Real Numbers

We demonstrate the use of these equivalence class based quotients with the Cauchy sequences defined in Section 3 to define the real numbers up to equality.

Because $=_c$ is an equivalence relation, we can define:

$$\mathbb{R} \coloneqq cauchy/=_c$$

All of our functions over Cauchy sequences, $\leq_c$, addition, multiplication, and additive and multiplicative inverses, respect equivalence, and so we can use $lift$ to define them over $\mathbb{R}$.

To facilitate these definitions, we define a couple of helper functions that let us define functions over a quotient using a function of the underlying type. Given a function over the underlying type that respects the equivalence relation, map lifts it to a function over the quotient:

$$
\begin{aligned}
&map : (f : A \to A) \\
&\quad \to (\forall\ x\ y, R\ x\ y \to R\ (f\ x)\ (f\ y)) \\
&\quad \to A/R \to A/R \\
&map := unwrap\ (lift\ (\lambda a.return\ (mk\ (f\ a)))\ \_)
\end{aligned}
$$

We get a defining equation that is helpful in reasoning about *map*:

$$map\ f\ r\ (mk\ a) = mk\ (f\ a)$$

Our formalization also includes an analogous function *map*2 for mapping functions of two inputs over a quotient, which allows us to define addition and multiplication. Finally, using $lift$, we define an ordering over the quotiented reals $\leq_{\mathbb{R}}$.

Using these tools, we formalize the entire real numbers. The total ordering property still needs to be under a double negation:

$$(x\ y : \mathbb{R}) \to [x \leq_{\mathbb{R}} y \vee y \leq_{\mathbb{R}} x]$$

However, all of the other real number axioms are able to be proven constructively, without any double negations or nonconstructivity monads in the statements. On the underlying type of Cauchy sequences defined in Section 3, we only could derive the existence of the least upper bound under a double negation. However, under the quotient, we are able to derive it directly. This is possible because least upper bounds are by definition unique up to Cauchy equivalence.

Given $S : R \to Prop$, we directly construct an equivalence class for our quotient consisting of Cauchy sequences with the property of being a least upper bound:

$$
\begin{aligned}
&class : cauchy \to CProp \\
&class := \lambda\ lub : cauchy.[\\
&\quad is\_upper\_bound\ S\ (mk\ lub) \wedge \\
&\quad (\forall\ b', is\_upper\_bound\ S\ b' \to b' \leq_{\mathbb{R}} (mk\ lub))]
\end{aligned}
$$

Using the classical existence of a least upper bound for Cauchy sequences, we are able to prove the existence property required by the quotient type. The property of respecting the equivalence relation follows from the uniqueness of least upper bounds. Therefore, this construction demonstrates a principle for quotients that is analogous to the unique description principle that we get for the nonconstructivity monad: given a proof of the unique existence up to equivalence of a quotient element, we can get the quotient element. We are able to derive a constructive function that outputs a least upper bound directly:

$$
\begin{aligned}
&(S : \mathbb{R} \to Prop) \\
&\to [\exists r, Sr] && \text{(nonempty)} \\
&\to [\exists b, is\_upper\_bound\ S\ b] && \text{(bounded)} \\
&\to \sum_{lub:\mathbb{R}} is\_upper\_bound\ S\ lub \\
&\quad \wedge (\forall b', is\_upper\_bound\ S\ b' \to lub \leq_c b')
\end{aligned}
$$

Overall, our formalization of the real numbers using a quotient has all of the usual axioms of the real numbers stated without any caveat except that the trichotomy property is under a double negation.

5 General Recursion

There are a wide variety of techniques for working with general recursive functions in constructive type theory [9]. The function type $A \to [\![B]\!]$ derived from our nonconstructivity monad represents a much larger class of functions than merely those definable by general recursion, or in other words those corresponding to a program that may not terminate. However, our monad can also be used for this purpose.

To do so, we first define a type family `Prog` which describes a function defined by general recursion. A function $A \to$ `Prog` $A\ B$ represents a function from A to B which may call itself recursively. The `Ret` constructor represents a completed computation, while `Rec a rest` represents a recursive call with input a that will continue the computation with *rest*.

```
Inductive Prog (A B : Type) : Type :=
| Ret : B -> Prog A B
| Rec : A -> (B -> Prog A B) -> Prog A B.
```

Using one of these descriptions of a recursive functions, we can define an inductive relation which represents running the computation from a starting state `Prog A B` to a result of type `B`:

```
Inductive runProgR (A B : Type) (def : A -> Prog A B)
  : Prog A B -> B -> Prop :=
| retR : forall b, runProgR def (Ret b) b
| recR : forall a b1 b2 rest,
    runProgR def (def a) b1
    -> runProgR def (rest b1) b2
    -> runProgR def (Rec a rest) b2.
```

One could stop here and be content to represent recursive functions as relations. But we can combine these definitions with our nonconstructivity monad to represent generally recursive functions using the function type. We can define a function *runProg* which outputs $[\![option\ B]\!]$. This function is defined directly in terms of the definition of the nonconstructivity monad. This includes a subset of $option\ B$, which contains *Some b* if the function is defined on the input, and outputs *None* if not.

$$runProg : (A \to Prog\ A\ B) \to A \to [\![option\ B]\!]$$
$$runProg\ prog\ a :=$$
$$(\lambda\ ob.$$
$$\exists b, ob = Some\ b \wedge runProgR\ prog\ (prog\ a)\ b$$
$$\vee\ ob = None \wedge \neg \exists b, runProgR\ prog\ (prog\ a)\ b$$
$$, \_, \_)$$

The uniqueness proof requires a proof that *runProgR* is a function in the relational sense. This is proven by induction:

$$runProgR\ def\ p\ b1 \to runProgR\ def\ p\ b2 \to b1 = b2.$$

The existence proof is straightforward, as we are working under a double negation and can use the law of the excluded middle. Either the function has an output for the given input or it doesn't, and so either *Some b* where *b* is the output is in the set, or *None* is in the set.

We can define two theorems which describe how to run these recursive functions.

```
Theorem evalRet
  : runProgImpl def (Ret b) = Creturn (Some b).

Theorem evalRec
  : runProgImpl def (Rec a rest) =
    Cbind (runProgImpl def (def a)) (fun ob =>
    match ob with
    | Some b => runProgImpl def (rest b)
    | None => Creturn None
    end).
```

Using these defining functions as well as the first monad law, we can write a convenient tactic which automatically runs these functions defined by general recursion.

```
Ltac evaluate := repeat rewrite ?evalRet,
  ?evalRec, ?monadlaw1.
```

To give an example, we write a function based on the Collatz conjecture. The function inputs a number, and outputs the number of operation steps of the Collatz conjecture that are needed to bring the number to 1.

```
Definition collatz_prog (n : nat) : Prog nat nat :=
  if eqb n 1 then Ret 0 else
    Rec (if (eqb (modulo n 2) 0)
           then n / 2
           else (3 * n) + 1)
        (fun res => Ret (1 + res)).

Definition collatz : nat -> [[|option nat|]] :=
  runProg collatz_prog.
```

Using our evaluation tactic, we can automatically evaluate our function at an input:

```
Goal collatz 6 = Creturn (Some 8).
```

In Section 7, we outline many existing techniques for general recursion in type theory. Our monad is probably excessive for this purpose compared to many of these other techniques. Instead, its main strength over other monads is that it represents a larger class of mathematical functions, which we made good use of in Sections 3 and 4. This larger class does however include functions defined by general recursion.

6 Monad Proofs

In Section 2, we defined our nonconstructivity monad and listed several useful properties that it satisfies, which we made good use of in Sections 3, 4, and 5. In this section, we will describe the proofs of these properties. The full proofs are also given in our formalization.

First, recall the three parts of the definition of the nonconstructivity monad: the subset, the existence proof, and the uniqueness proof.

$$[\![T]\!] := \sum_{S:T \to CProp} [\exists t : T, S\ t] \wedge (\forall x \forall y, S\ x \to S\ y \to [x = y])$$

As we write our definitions, we will often leave the proofs of existence and uniqueness as underscores, and then describe them in text.

First, we need to define the monad operations; *return* gives the singleton subset for a value, while *bind* maps a subset *S* and a family of subsets *f* to the union of the image of *f* over *S*. We omit for the moment the uniqueness and existence proofs:

$$return : T \to [\![T]\!]$$
$$return\ t := (\lambda\ t'.[t = t'], \_, \_)$$
$$bind : [\![A]\!] \to (A \to [\![B]\!]) \to [\![B]\!]$$
$$bind\ (S, existence_A, unique_A)\ f :=$$
$$(\lambda\ b.[\exists a, S\ a \wedge proj_1\ (f\ a)\ b], \_, \_)$$

For *return*, the omitted proofs of existence and uniqueness are both straightforward, since *t* is the single unique element. For *bind*, uniqueness follows from the uniqueness property of the inputs: there is only one *a* such that *S a*, and then there is only one *b* such that $proj_1\ (f\ a)\ b$. Existence follows from the existence properties of the inputs. We write out *bind*'s existence proof completely to give a demonstration of reasoning using the double negation monad:

$$
\begin{aligned}
&bind\_existence : \ [\exists b,\ [\exists a,\ S_A\ a \wedge f\ a\ b]] \\
&bind\_existence := \\
&\qquad bind_P\ existence_A\ \lambda(a,\ S_A a). \\
&\qquad \texttt{let}\ (S_B,\ existence_B,\ unique_B) = f\ a\ \texttt{in} \\
&\qquad bind_P\ existence_B\ \lambda(b,\ S_B b). \\
&\qquad return_P\ (b,\ return_P\ (a,\ S_A a,\ S_B b))
\end{aligned}
$$

We can also prove that *return* is injective. We only get this up to a double negation, but it is still useful for refuting equalities between values under the monad.

$$return\ x = return\ y \to [x = y]$$

We can prove that our monad does indeed satisfy the three monad laws:

$$
\begin{aligned}
bind\ (return\ x)\ f &= f\ x && \text{(left identity)} \\
bind\ x\ return &= x && \text{(right identity)} \\
bind\ (bind\ x\ f)\ g &= && \text{(associativity)} \\
&\quad bind\ x\ (\lambda\ x.\ bind\ (f\ x)\ g)
\end{aligned}
$$

Next, we will prove the induction principle for the nonconstructivity monad. In fact, we can prove classically that all elements of T are equivalent to $return\ t$ for some $t : T$. If $x : [\![T]\!]$, by its existence property, we get $[\exists t : T, proj_1\ x\ t]$. Using functional and propositional extensionality, as well as the uniqueness property of x, we can then show that x is equal to $return\ t$:

$$(x : [\![T]\!]) \to [\exists t : T, x = return\ t \wedge proj_1\ x\ t]$$

Using this property, we can prove the induction principle.

$$
\begin{aligned}
ind : (Q : [\![T]\!] \to Prop) &\to (x : [\![T]\!]) \\
&\to (\forall t, proj_1\ x\ t \to [Q\ (return_P\ t)]) \\
&\to [Q\ x]
\end{aligned}
$$

The conclusion of the principle is under the double negation monad, and so the existence property can be unwrapped from its double negation using $bind_P$. The rest of the proof is just a matter of rewriting the conclusion using the resulting equality, and then specializing the premise to t.

We mentioned the following property that allows us to remove a double negation around an equality between two elements of the monad:

$$\forall a, b : [\![T]\!], [a = b] \to a = b$$

This proof makes use of the fact that the subsets in the definition of the nonconstructivity monad output *CProp*s rather than merely any proposition. By functional and propositional extensionality, we ultimately need to prove that for an element $t : T$, $proj_1\ a\ t \iff proj_1\ b\ t$. Because these two values are *CProp*s, we can use $bind_P$ in their derivations to remove the double negation from $[a = b]$.

Finally, we will need a way to formulate propositions that depend on nonconstructive values. If we define a proposition under the monad, we can convert it to a normal proposition by checking if it is equal to $return\ True$:

$$
\begin{aligned}
&toProp : [\![Prop]\!] \to Prop \\
&toProp\ P := (P = return\ True)
\end{aligned}
$$

Reasoning with *toProp* requires one last property of our monad. Using the monad laws and the induction principle, we will often be able to reduce the nonconstructive proposition to the form $return\ P$. Then, we can use the following property to relate this value to a classical proposition:

$$toProp\ (return\ P) = [P]$$

In our Rocq formalization, in addition to a formalization of these proofs, we have written a couple of automated tactics that apply some of these reasoning principles.

First, we have a tactic `classical_auto` which automatically simplifies several things. It:

- rewrites by the first monad law
- simplifies $toProp\ (return\ P)$ into $[P]$
- uses $bind_P$ to remove any double negations around hypotheses, if there is a double negation around the conclusion.

These several automated steps can often chain together: the monad law can reduce an expression to $return\ x$, which can be converted to a double negation if under a *toProp* and then finally unwrapped into a normal proposition.

We also have a tactic `classical_induction` which applies our monad's induction principle with analogous syntax to Rocq's built-in induction tactic for inductive types. Given some value $t : [\![T]\!]$, `classical_induction t` will introduce a $x : T$ in context and replace t with $return\ x$.

7 Related Work

There is a substantial amount of existing work which seeks to represent larger classes of functions in constructive type theory.

7.1 General Functions in Type Theory

The partial function classifier monad. Most closely related to our monad $[\![T]\!]$ is the partial function classifier monad, first defined in type theory in Escardó and Knapp [16]. Their monad is defined in homotopy type theory and they give several equivalent formulations, but their equation (5) is closest to our presentation. Rendering their monad $\mathcal{L}$ in the notation of this paper it is equivalent to:

$$\mathcal{L}\ T = \sum_{S:T \to Prop} (\forall xy, Sx \to Sy \to x = y)$$

So it differs from ours in the lack of the existence condition $[\exists x, Sx]$, as well as in the lack of double negations. Therefore, $\mathcal{L}\ T$ has an extra empty element and is analogous to $[\![\texttt{option}\ T]\!]$. A function $A \to \mathcal{L}\ B$ represents partial functions from A to B that may be undefined on some inputs. This makes it ideal for defining partial functions by general recursion, which is one aim of their paper. However, it is less well suited to representing total functions, as it would require an extra side condition to restrict the function to always output a value. Our monad is more useful to represent the traditional mathematical notion of a total function as we investigated in Sections 3 and 4.

Monads for Nontermination. There are various monads in the literature which allow general recursion. These monads correspond to possibly nonterminating programs, but not any larger class of mathematical functions. These include the coinductive delay monad [10] and other coinductive monads [9, 12], and monads based on fuel [9, 12]. One disadvantage of these monads is their behavior with respect to the equality type, and recent work addresses this problem for the delay monad [11].

Axiomatic Approaches. One approach to model general mathematical functions in type theory is to introduce additional axioms that add more functions to the existing function type. A general drawback of these approaches is that introducing these axioms breaks canonicity. This problem can at least partial be solved with an effect typing system that tracks which definitions refer to the axioms, as in Lean [15].

Introducing the classical axiom of choice extends the function type to include all mathematical functions. The Lean theorem prover [15] for example formulates the axiom as $\texttt{Nonempty}\ T \to T$, given $\texttt{Nonempty} : Type \to Prop$.

Another approach is to use the more limited definite description axiom [2]. This along with the law of the excluded middle can be used to derive fixpoints [4]. This approach is very similar to our monad, in that our monad also gives access to these two principles.

Extensions to Type Theory for General Recursion. Prior work has investigated the addition of extra features to type theory itself to allow the user to express potentially nonterminating programs. Bove and Capretta [8] augment type theory with a new type former for partial computations. Constable and Smith [14] introduce a partiality type former. Weirich et al. [19] add labels to types which distinguish whether they allow nontermination.

7.2 Related work on Real Numbers

The real numbers have been formalized in a wide variety of theorem provers using a variety of constructions; see Boldo et al. [7] for a survey. In a classical logic these constructions can follow the traditional proofs, but in a constructive logic these traditional proofs don't yield an object with all of the usual real number axioms.

Constructive Real Numbers. Although the classical real numbers can't be directly defined in constructive type theory, as described in Section 3, there is different notion of real numbers that can called the *constructive real numbers*.

The constructive reals lack some of the properties of the classical real numbers, although a significant amount of analysis can be developed for them [5, 6]. See Section 2.4 of Ciaffaglione [13] for an overview of their history.

The exact properties of the constructive reals depends on the method of their construction. The constructive Dedekind reals [17] lack only the trichotomy property; similar to our construction it can only be derived under a double negation. Most constructions of reals in constructive mathematics also lack the Dedekind completeness property, instead proving the archimedian and Cauchy completeness properties. Constructive reals with these properties have been formalized as streams [13] and Cauchy sequences [3]. They have also been defined as Cauchy sequences using a higher inductive type [17].

8 Conclusion

In this paper, we have demonstrated a new monad to simulate a classical definite description principle in a constructive type theory. Using this along with the well known monad of double negation, we demonstrated some nonconstructive mathematics in a constructive type system. However, the existence of these monads does not necessarily mean that there is no need to include classical axioms in a formal system — monads can be annoying to work with. Still, the existence of these techniques further blurs the line between classical and constructive mathematics. In the future, we intend to continue studying how to simulate features in languages lacking them and exploring the implications to language design.

References

[1] [n. d.]. Mathlib. https://leanprover-community.github.io/
[2] [n. d.]. The Rocq Stdlib, Stdlib.Logic.Description. https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Logic.Description.html
[3] [n. d.]. The Rocq Stdlib, Stdlib.Reals.Cauchy.ConstructiveCauchyReals. https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Reals.Cauchy.ConstructiveCauchyReals.html
[4] Yves Bertot and Vladimir Komendantsky. 2008. Fixed point semantics and partial recursion in Coq. In *Proceedings of the 10th international ACM SIGPLAN conference on Principles and practice of declarative programming* (Valencia Spain). ACM, New York, NY, USA.

[5] Errett Bishop. 1967. Foundations of constructive analysis. (1967). doi:10.2307/2272421

[6] E Bishop and Douglas S Bridges. 1985. *Constructive Analysis*. Springer, Berlin, Germany.

[7] Sylvie Boldo, Catherine Lelay, and Guillaume Melquiond. 2016. Formalization of real analysis: a survey of proof assistants and libraries. *Mathematical Structures in Computer Science* 26, 7 (2016), 1196–1233. doi:10.1017/S0960129514000437

[8] Ana Bove and Venanzio Capretta. 2008. A Type of Partial Recursive Functions. In *Theorem Proving in Higher Order Logics*, Otmane Ait Mohamed, César Muñoz, and Sofiène Tahar (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 102–117.

[9] Ana Bove, Alexander Krauss, and Matthieu Sozeau. 2016. Partiality and recursion in interactive theorem provers – an overview. *Math. Struct. Comput. Sci.* 26, 1 (Jan. 2016), 38–88.

[10] Venanzio Capretta. 2005. General Recursion via Coinductive Types. *CoRR* abs/cs/0505037 (2005). arXiv:cs/0505037 http://arxiv.org/abs/cs/0505037

[11] James Chapman, Tarmo Uustalu, and Niccolò Veltri. 2019. Quotienting the delay monad by weak bisimilarity. *Mathematical Structures in Computer Science* 29, 1 (2019), 67–92. doi:10.1017/S0960129517000184

[12] Adam Chlipala. 2013. *Certified programming with dependent types*. MIT Press, London, England. http://adam.chlipala.net/cpdt/html/GeneralRec.html

[13] Alberto Ciaffaglione. 2003. *Certified reasoning on real numbers and objects in co-inductive type theory*. Ph. D. Dissertation. éditeur inconnu.

[14] Robert L Constable and Scott Fraser Smith. 1987. *Partial objects in constructive type theory*. Technical Report. Cornell University.

[15] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. *The Lean Theorem Prover (System Description)*. Springer International Publishing, 378–388. doi:10.1007/978-3-319-21401-6_26

[16] Martín H. Escardó and Cory M. Knapp. 2017. Partial Elements and Recursion via Dominances in Univalent Type Theory. In *26th EACSL Annual Conference on Computer Science Logic (CSL 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 82)*, Valentin Goranko and Mads Dam (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 21:1–21:16. doi:10.4230/LIPIcs.CSL.2017.21

[17] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. https://homotopytypetheory.org/book, Institute for Advanced Study.

[18] Philip Wadler. 1992. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '92 (POPL '92)*. ACM Press, 1–14. doi:10.1145/143165.143169

[19] Stephanie Weirich, Vilhelm Sjoberg, and Chris Casinghino. 2013. Combining Proofs and Programs in a Dependently Typed Language. (01 2013).